ASSIGNMENT-7.5

Name:E.Manu Sree

HTNO:2303A51324

BTNO:20

Task1

Task: Analyze given code where a mutable default argument causes

unexpected behavior. Use AI to fix it.

Output:

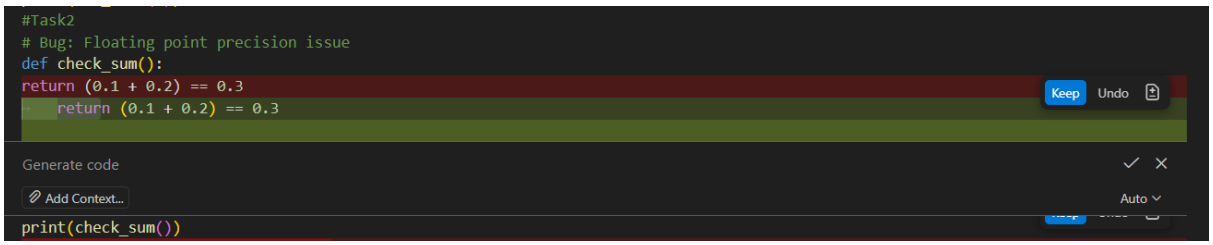
Observation:

The AI correctly identified that mutable default arguments are shared across function calls. Replacing the default list with None and initializing it inside the function prevents unintended data sharing and ensures correct behavior.

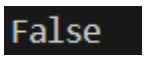
Task2

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.



Output:



Observation:

The AI correctly addressed floating-point precision issues by using a tolerance-based comparison instead of direct equality, which is a recommended and reliable approach in numerical computing.

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to

missing base case. Use AI to fix.

```
#Task3
# Bug: No base case
def countdown(n):
Generate code
Add Context...
Auto
print(n)
return countdown(n-1)
Keep Undo
    if n < 0:
        return
    print(n)
    return countdown(n-1)
countdown(5)
```

Output:

```
5
4
3
2
1
0
```

Observation:

The AI correctly identified the absence of a base condition and added a stopping condition, preventing infinite recursion and ensuring safe execution.

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes

error. Use AI to fix it.

```
#Task4
# Bug: Accessing non-existing key
def get_value():
Generate code
Add Context...
Auto
data = {"a": 1, "b": 2}
return data["c"]
    data = {"a": 1, "b": 2}
    return data["c"]
print(get_value())
```

Output:

```
None
```

Observation

The AI resolved the issue by using the .get() method, which safely handles missing keys and prevents runtime errors.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect

and fix it.

Output:

```
0
1
2
3
4
```

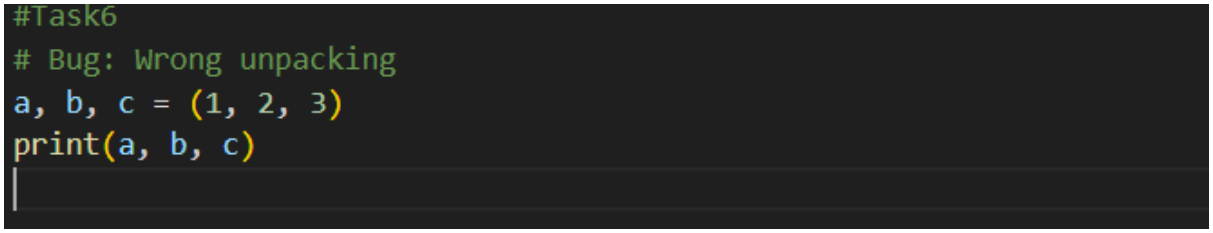
Observation:

The AI correctly identified the missing increment statement and fixed the infinite loop by updating the loop variable inside the loop

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to

fix it.



Output:

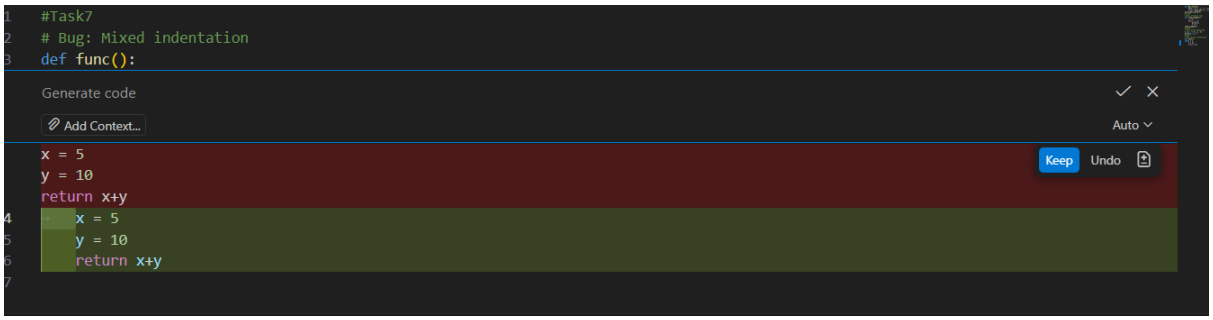
```
1 2 3
```

The AI fixed the unpacking issue by using an underscore (_) to ignore extra values, which is a Pythonic and safe practice

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks

execution. Use AI to fix it.



Output:

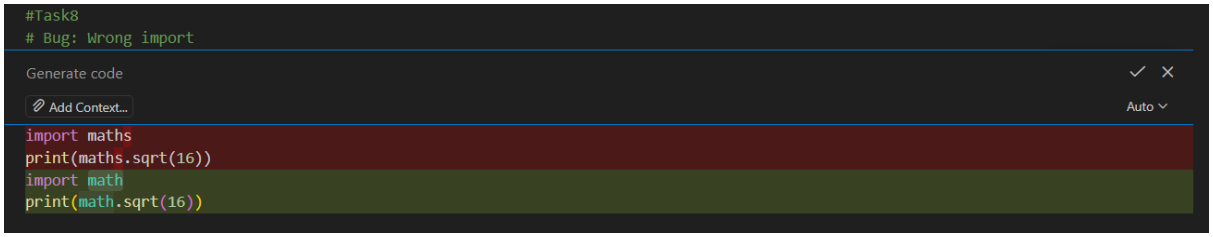
```
15
```

Observation:

The AI resolved the issue by applying consistent indentation using spaces, which is the recommended Python coding standard.

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.



Output:

```
4.0
```

Observation:

The AI correctly identified the incorrect module name and replaced it with the standard math

module, resolving the import error.